IMPROVING INSTRUCTION-FOLLOWING IN LANGUAGE MODELS THROUGH ACTIVATION STEERING

Alessandro Stolfo[1*] **Vidhisha Balachandran**[2] **Safoora Yousefi**[2]
Eric Horvitz[2] **Besmira Nushi**[2]
[1]ETH Zürich [2]Microsoft Research

arXiv:2410.12877v2 [cs.CL] 14 Apr 2025

ABSTRACT

The ability to follow instructions is crucial for numerous real-world applications of language models. In pursuit of deeper insights and more powerful capabilities, we derive instruction-specific vector representations from language models and use them to steer models accordingly. These vectors are computed as the difference in activations between inputs with and without instructions, enabling a modular approach to activation steering. We demonstrate how this method can enhance model adherence to constraints such as output format, length, and word inclusion, providing inference-time control over instruction following. Our experiments across four models demonstrate how we can use the activation vectors to guide models to follow constraints even without explicit instructions and to enhance performance when instructions are present. Additionally, we explore the compositionality of activation steering, successfully applying multiple instructions simultaneously. Finally, we demonstrate that steering vectors computed on instruction-tuned models can transfer to improve base models. Our findings demonstrate that activation steering offers a practical and scalable approach for fine-grained control in language generation. Our code and data are available at https://github.com/microsoft/llm-steer-instruct.

1 INTRODUCTION

Instruction-following capabilities of large language models (LLMs) have enhanced their practical applications for real-world usage. These advances are powered by instruction-tuning methods (Ouyang et al., 2022; Bai et al., 2022; Wei et al., 2022; Sanh et al., 2022; Chung et al., 2024), which align the model's responses with user objectives, addressing the gap between pre-training and end-user needs (Askell et al., 2021). Instruction tuning allows users to specify constraints on attributes like format, tone, or length, which direct the model's behavior and output (Zhou et al., 2023b; Zhang et al., 2023; Lou et al., 2024). Gaining a deeper understanding of how LLMs internally represent and follow these instructions is essential for developing more controllable and reliable models.[1]

In this paper, we use a mechanistic method to investigate how language models internally represent various instructions and use these representations to influence and control the model's behavior. Prior research has shown that vector representations can be computed for tasks learned in context (Hendel et al., 2023; Todd et al., 2024; Liu et al., 2024) and various stylistic and semantic input features (Zou et al., 2023; Azaria & Mitchell, 2023; Zheng et al., 2024; Templeton et al., 2024; Marks & Tegmark, 2024, *inter alia*). These representations can be used for activation steering (Subramani et al., 2022): directly intervening on the model's activations to guide the generation. This approach has been successfully applied to control text attributes such as honesty (Li et al., 2023; Qiu et al., 2024; Zou et al., 2023), sentiment (Tigges et al., 2024), output style and tone (Turner et al., 2023; Liu et al., 2024; Scalena et al., 2024; von Rütte et al., 2024), harmfulness (Arditi et al., 2024; Wang & Shu, 2024), and sycophancy (Panickssery et al., 2024; van der Weij et al., 2024).

However, user instructions in generative tasks can be more complex and involve multiple parameters that need to be attended to and satisfied during generation (Sun et al., 2023). For example, users may

*Work done while at Microsoft Research. Corresp. to stolfoa@ethz.ch and besmira.nushi@microsoft.com.

[1]We use *instruction* to refer to specific constraints that can be added during interactive sessions with LLMs in a modular way to modify or extend a *base* question or request. We further elaborate on this definition in §2.

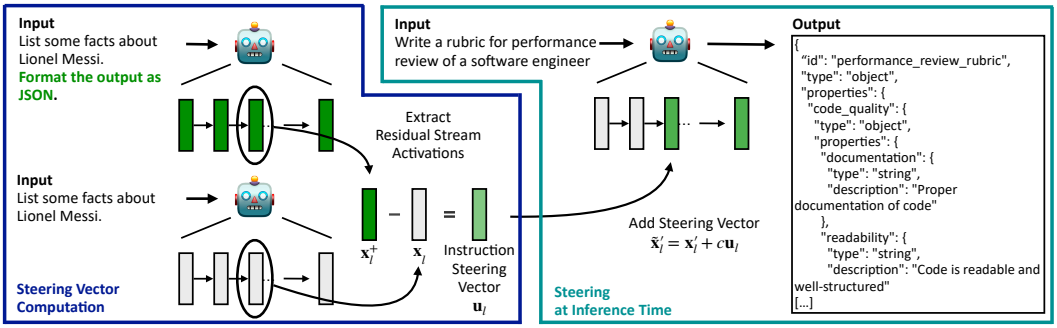

Figure 1: **Instruction Steering Process.** Steering vectors are computed as the difference in residual stream activations between inputs with and without the instruction. These vectors are then applied during inference to adjust the model's activations, guiding it to follow the desired instruction.

ask for a response to include bullet lists, have a certain number of sentences, or include/exclude specific content. The complexity of such instructions stems from the high number of possible variations, making it impractical to generate post-training data for all scenarios. In the quest for finding more efficient methods for controlling instruction following and guided by previous research in language model representations, we pose the question: Can we efficiently extract vector representations that encode and control specific instruction-following behavior?

We investigate this using a contrastive, additive steering method (Turner et al., 2023; Panickssery et al., 2024) that computes the difference in activations between inputs with and without an instruction (Figure 1, left), and applies this difference to guide the model to better follow instructions on new inputs (Figure 1, right). While previous work focused on high-level stylistic aspects, we target more diverse, verifiable instructions (Zhou et al., 2023a) that complement the base request's semantics (e.g., formatting, length constraints). These instructions are lower-level and more specific, with multiple possible instantiations (e.g., "Do not mention the word {keyword}"). It remains unclear whether models represent such instructions linearly and whether these representations can be used to reliably elicit specific behaviors, which would enable finer control of LLM outputs.

We conduct experiments using the Phi-3 (Abdin et al., 2024), Gemma 2 2B and 9B (Gemma Team, 2024), and Mistral 7B (Jiang et al., 2023) models, focusing on three types of instructions: output format (§3), output length (§4), and the inclusion/exclusion of specific words (§5). Our results on the IFEval dataset (Zhou et al., 2023a) provide evidence that vector representations can encode a wide range of instructions and enhance the model's instruction-following performance. Notably, we demonstrate that activation steering not only helps models follow constraints when no instruction is provided in the input, but it can also reinforce instruction adherence even when instructions are explicitly present, potentially countering *instruction drift* (Li et al., 2024). Furthermore, we show that it is possible to simultaneously steer for multiple constraints, such as controlling both format and length (§6). Finally, we present compelling evidence that cross-model steering–using vectors computed on an instruction-tuned model to steer a base model–is effective and can sometimes yield better adherence than same-model steering (§7), suggesting new possibilities for transferring task-specific skills across models, similar to "task arithmetic" (Ilharco et al., 2023).

Our work represents an important step toward operationalizing techniques from mechanistic interpretability to achieve practical improvements in scenarios with real-world utility.

2 Steering for Instruction Adherence

In this section, we define the types of instructions we consider (§2.1), introduce the methodology used to compute the vectors and the steering procedure (§2.2), and describe the experimental setup, including the data, metrics, and evaluation details (§2.3).

2.1 Types of Instructions

The concept of instruction-following has been used to describe the broad capability of a model to answer any zero-shot query (Wei et al., 2022; Sanh et al., 2022). However, following prior work

(Zhou et al., 2023a;b; Sun et al., 2023), we adopt a more specific definition of instruction that refers to a constraint applied to a particular aspect of the model's output. These constraints are self-contained, modular, and can be imposed on various base queries. This definition decouples the ability of the model to follow the instruction from its factual knowledge and domain-specific skills. In our work, we focus on three specific types of instructions:

1. **Format instructions**, which dictate how the output should be presented. For instance, the model may be asked to produce responses in a specific format (e.g., "Provide the answer in JSON format") or highlight parts of the response in a particular way (e.g., "Use asterisks to emphasize at least two sections of the answer").
2. **Length instructions**, which specify the desired length of the output (e.g., "Answer using at most three sentences").
3. **Word-specific instructions**, which control the inclusion or exclusion of specific words or phrases in the output (e.g., "Do not include the word *AI* in the answer").

2.2 Steering Procedure

Activation (or representation) engineering involves constructing vectors of activation values which cause desired changes to output text when added to the forward passes of a frozen LLM (Zou et al., 2023). To identify a direction in the model's residual stream[2] that encodes information about a specific instruction, we use a technique called *difference-in-means* (Belrose, 2023). This method effectively isolates key feature directions in the model's internal activations (Marks & Tegmark, 2024; Tigges et al., 2024) and has been used to control various behaviors such as refusal and sycophancy (Arditi et al., 2024; Panickssery et al., 2024). We adapt this method to support finer-grained instructions with multiple possible instantiations (e.g., varying length, varying words to include and exclude). The process involves pairing two versions of the same request: one with only the base query (e.g., "List some facts about Lionel Messi") and another that additionally includes the instruction we want to represent (e.g., "List some facts about Lionel Messi, ensuring the output is valid JSON").

Let us denote these two inputs by x (base query) and x^+ (base query with instruction), and consider a set of N such pairs (x_i, x_i^+), $i \in \{1, \ldots, N\}$. Let $\mathbf{x}_{i,l}, \mathbf{x}_{i,l}^+ \in \mathbb{R}^{d_{\text{model}}}$ be the values of the residual stream vector on the two queries at the last token of the input at layer $l \in \{1, \ldots, L\}$. We isolate the internal representation corresponding to the instruction by computing the difference in the residual stream vectors between the paired inputs. More formally, we compute a vector $\mathbf{u}_l \in \mathbb{R}^{d_{\text{model}}}$ representing the steering direction at layer l for a given instruction as:

$$\mathbf{u}_l = \frac{\mathbf{v}_l}{\|\mathbf{v}_l\|}, \text{ where } \mathbf{v}_l = \frac{1}{N} \sum_i^N \left(\mathbf{x}_{i,l}^+ - \mathbf{x}_{i,l}\right). \tag{1}$$

Averaging over different base queries allows us to capture the activation values most closely associated with the instruction, independent of the base query. The computation of the steering direction is carried out using the representations at the last token of the input, which effectively encapsulate the model's behavior not only for the next-token-prediction task but also for the entire generation that follows (Todd et al., 2024; Scalena et al., 2024).

After identifying the steering direction, we compute the steering vector by re-scaling the unit vector $\mathbf{u}_l$ by a coefficient, $c \in \mathbb{R}$. For format instructions, we use a systematic scaling approach where the value of c is selected to ensure that the residual stream activations are mapped to their mean value on inputs that contain the instruction in question. In particular, during a forward pass on a new example with residual stream values $\mathbf{x}' \in \mathbb{R}^{L \times d_{\text{model}}}$ at a given token position, we compute:

$$c = \bar{z} - \mathbf{x}_l'^{\mathrm{T}} \mathbf{u}_l, \text{ where } \bar{z} = \frac{1}{N} \sum_i^N \mathbf{x}_{i,l}^{+\mathrm{T}} \mathbf{u}_l. \tag{2}$$

This dynamic adjustment allows the model to effectively incorporate the constraint without over- or under-correcting its behavior. For length instructions, which have a more continuous nature, we experiment and show results with different values of c, illustrating their impact on the model's

[2]The model's residual stream is the per-token hidden state of dimensionality d_{model} consisting of the sum of all previous component outputs (Elhage et al., 2021).